

CS 35L Software Construction

Lecture 11 – Security & Authentication

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department



Why should we care about Security?

- Security **breaches** in 2024 were **up 75% year-over-year**, with organizations facing an average of 1876 attacks per quarter [1]
- Data breaches **cost \$4.4M each** on average [2]

[1] <https://www.accenture.com/content/dam/accenture/final/accenture-com/document-3/State-of-Cybersecurity-report.pdf>

[2] <https://www.ibm.com/reports/data-breach>

The CIA Triad

The Three Main Security Attributes

Confidentiality

Sensitive data must be **accessed by authorized users only**

Integrity

Sensitive data must be **modifiable by authorized users only.**

The system **maintains the accuracy, consistency, and trustworthiness of data over its entire lifecycle.**

Availability

Critical services must be **available when needed by clients**

Which Security Attribute(s) is/are affected?

2016: Ransomware takes Hollywood Presbyterian Medical Center offline

\$3.6M demanded by attackers

Malware locks systems by encrypting files and demanding ransom to obtain the decryption key

Availability
Patient records are unavailable

Integrity
Encryption modified the files

See <https://www.csoonline.com/article/554745/ransomware-takes-hollywood-hospital-offline-36m-demanded-by-attackers.html>

Which Security Attribute(s) is/are Affected?

2017 Equifax Data Breach

Sensitive **personal data of 148 million Americans** extracted (names, home addresses, phone numbers, dates of birth, social security numbers, and driver's license numbers)

Cost: **\$1.38 billion total**, including settlements, regulatory fines, and mandatory security improvements.

Confidentiality
Encryption modified
the files

See <https://www.breachsense.com/blog/equifax-data-breach/>

SQL Injection Attack

What does it do? Talk to your neighbor(s)

```
name = get_user_input("username");
pass = get_user_input("userpassword");
sql = ('SELECT * FROM Users '
      'WHERE Name = "' + name + '"'
      'AND Pass = "' + pass + '"')
user = db.execute_query(sql);
login(user) if user else retry()
```

How can we break into this system without knowing the password of a user?
Talk to your neighbor(s)

Returns the user Tobias if his password is password1234.
Otherwise returns nothing.

Username: Tobias

Password: password1234

```
SELECT *
FROM Users
WHERE Name = "Tobias"
AND Pass = "password1234"
```

Username: Tobias

Password: " or ""="

```
SELECT *
FROM Users
WHERE Name = "Tobias"
AND Pass = "" or ""="
```

""="" is always true.
Hence, Pass = ""
or ""="" is also
always true

SQL Injection (SQLi) is One of the Most Common Software Vulnerabilities

- **6.7%** of all vulnerabilities found in open-source projects are SQL Injection (2023)

1998

SQLi attack was **first described** publicly

2010-2021

OWASP ranks SQLi **#1 web app security vulnerability**

2011

Sony PlayStation breach compromises passwords & data of over **77 million users**

2023

SQLi in **MOVEit** transfer app affects **thousands of** companies worldwide

If SQL Injection is so common then it is probably very hard to fix, right?

RIGHT????

See https://www.w3schools.com/sql/sql_injection.asp & <https://www.aikido.dev/blog/the-state-of-sql-injections>

Use Prepared Statements and Parameterized Queries to Protect against SQL Injection (SQLi)

- 6.7% of all vulnerabilities found in open-source projects are SQL Injection (2023)



```
name = get_user_input("username");
pass = get_user_input("userpassword");
sql = ('SELECT * FROM Users '
      'WHERE Name = @0 '
      'AND Pass = @1')
user = db.execute_query(sql, name, pass);
login(user) if user else retry()
```

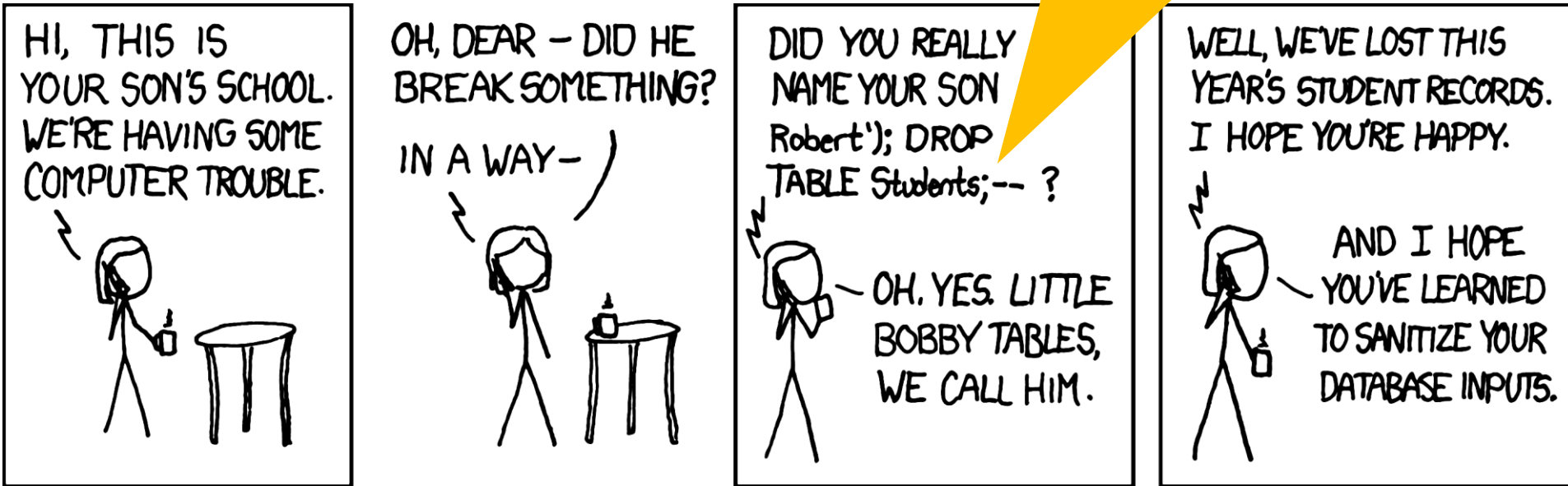
Parameters referenced by their index. The **syntax** varies by DBMS

The DBMS is **sanitizing the input parameters** to prevent injection attacks

See https://www.w3schools.com/sql/sql_injection.asp & <https://www.aikido.dev/blog/the-state-of-sql-injections>

Explain this Comic

'); ends the SQL query
DROP TABLE removes the entire student database table
; -- ignores the remainder of the original query to avoid syntax errors



Source: <https://xkcd.com/327/>

Quiz: Which Security Attribute(s) is/are Affected by SQL Injection Attacks?

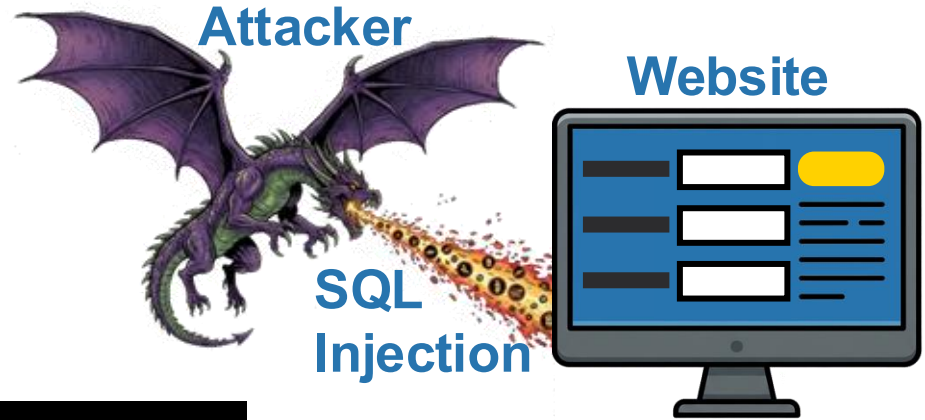
Confidentiality

Attackers can **read sensitive data** from the database

Integrity

Attackers can **modify, insert, or delete** data in the database

```
name = get_user_input("username");
pass = get_user_input("userpassword");
sql = ('SELECT * FROM Users '
      'WHERE Name = @0 '
      'AND Pass = @1')
user = db.execute_query(sql, name, pass);
login(user) if user else retry()
```



Rare in practice

Availability

Attacks can **disrupt database operations** (e.g., dropping tables, causing system crashes)

How Could We Try to Attack this Website?



Tobias Dürschmid

November 5 at 4 PM

UCLA students are by far the best students. Go Bruins!



David Smallberg, Sandra Batista, and 999 others

5 comments

Like

Comment

Share



Paul Eggert
Facts



Write a comment ...

Cross-Site Scripting (XSS) Attack (JavaScript Injection)



Tobias Dürschmid

November 5 at 4 PM

UCLA students are by far the best students. Go Bruins!



David Smallberg, Sandra Batista, and 999 others

5 comments

Like

Comment

Share



Paul Eggert

Facts



```
<script>alert(USC IS BETTER!!!)</script>
```

Cross-Site Scripting (XSS) Attack (JavaScript Injection)



Tobias Dürschmid

November 5 at 4 PM

UCLA students are by far the best students. Go Bruins!



David Small

6 comments

Like

Share



Attacker

Alert

USC IS BETTER!!!

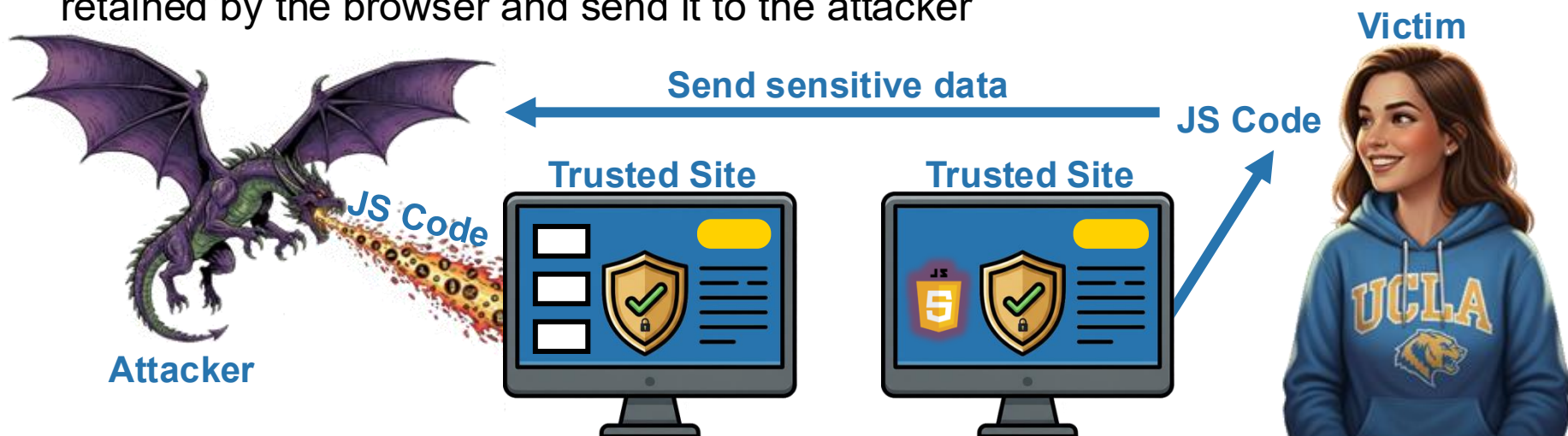
OK



Write a comment ...

Cross-Site Scripting (XSS) Attack (JavaScript Injection)

- An attacker **injects JS code** into a trusted site
- Browser of the **victim executes the JS code** from a trusted site
- JS code can access cookies, session tokens, or other sensitive information retained by the browser and send it to the attacker



Quiz: Which Security Attribute(s) is/are Affected by SQL Injection Attacks?

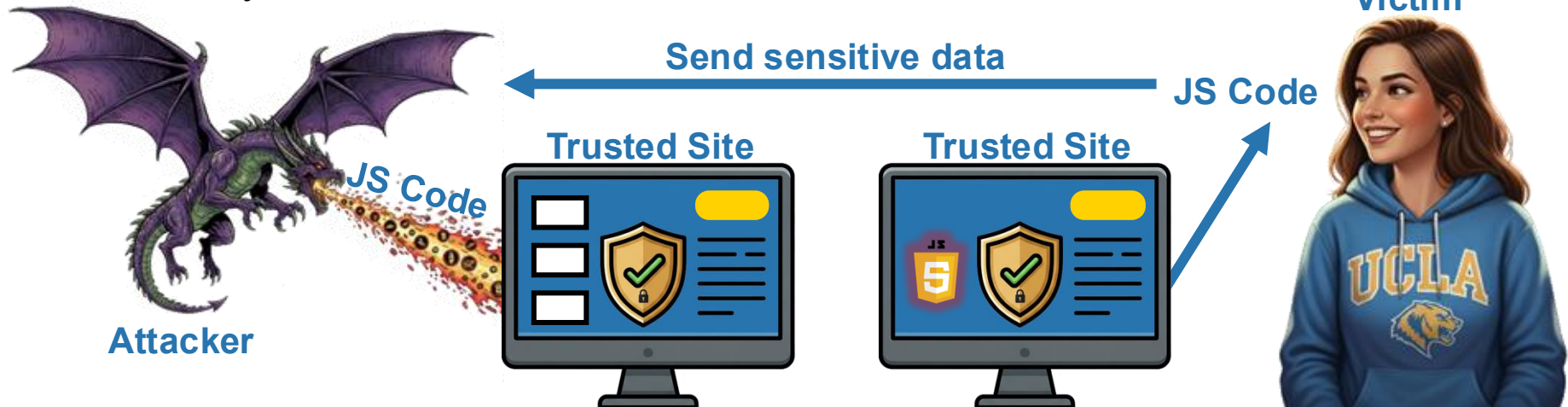
Confidentiality

Attackers can **read sensitive data** from the client & might gain access to **privileged website content**

retained by the browser and send it to the attacker

Integrity

Attackers can **modify webpage content** & might submit **unauthorized requests**



Prominent Examples of XSS Attacks

2010 Twitter Worm

Twitter's parsing process failed to properly **sanitize** or **escape** certain characters (like `onmouseover`) in a tweet.

- Pranks (e.g., displaying rainbow colored text or pop-up alerts)
- Redirecting users to malicious third-party websites

2018 British Airways Data Breach

Names, addresses, credit card numbers and card security codes stolen from almost 250,000 customers

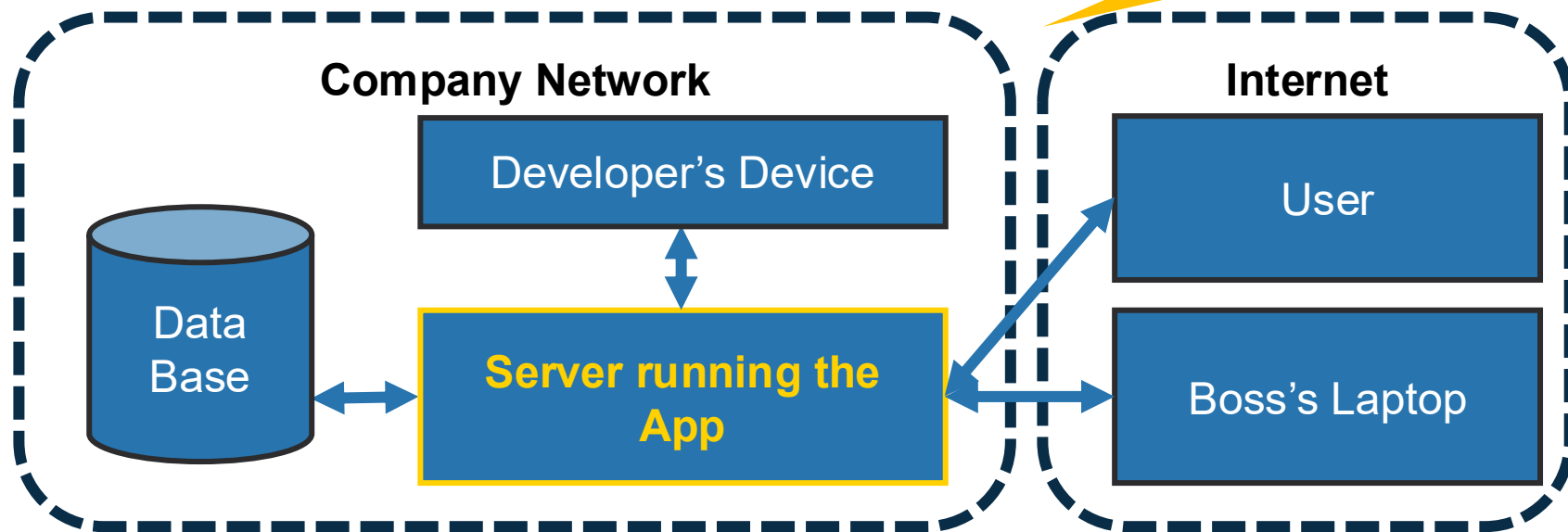
JS injected into the webpage **re-directed users to a bogus site**

See <https://www.theguardian.com/technology/blog/2010/sep/21/twitter-hack-explained-xss-javascript> & https://en.wikipedia.org/wiki/British_Airways_data_breach

Security Design Principle: Zero Trust Principle & Trust Boundaries

- Users and devices **should not be trusted** by default
- Any **input** might be malicious, so **sanitize all inputs**

Where to draw the trust boundary?



Security Through Obscurity

Should NOT be your Primary Approach to Security

Security Through Obscurity

Concealing the **details** or **mechanisms** of a system to **enhance its security**

Weakness

The implementation details of the app can be **discovered**, which opens vulnerabilities

Don't just hide the key in the flowerpot

"This house is secure because nobody knows that the key is in the flowerpot"
– Security Through Obscurity



Carefully apply The Open Design Principle

Security Through Obscurity

Concealing the details or mechanisms of a system to **enhance its security**

Open Design Principle

Attackers shouldn't be able to break into a system simply by **understanding how it works**. Use robust, public security mechanisms.

Public Scrutiny

Full transparency allows independent experts, researchers, and the security community to **examine, test, and critique the design** for potential vulnerabilities.

Complementary Obscurity

Hide implementation details & configuration details (e.g., version of a particular framework, data base, server configuration) to make it **harder** to exploit known vulnerabilities

When to Apply Public Scrutiny and When to Apply Complementary Obscurity?

Public Scrutiny

Full transparency allows independent experts, researchers, and the security community to **examine, test, and critique the design** for potential vulnerabilities.

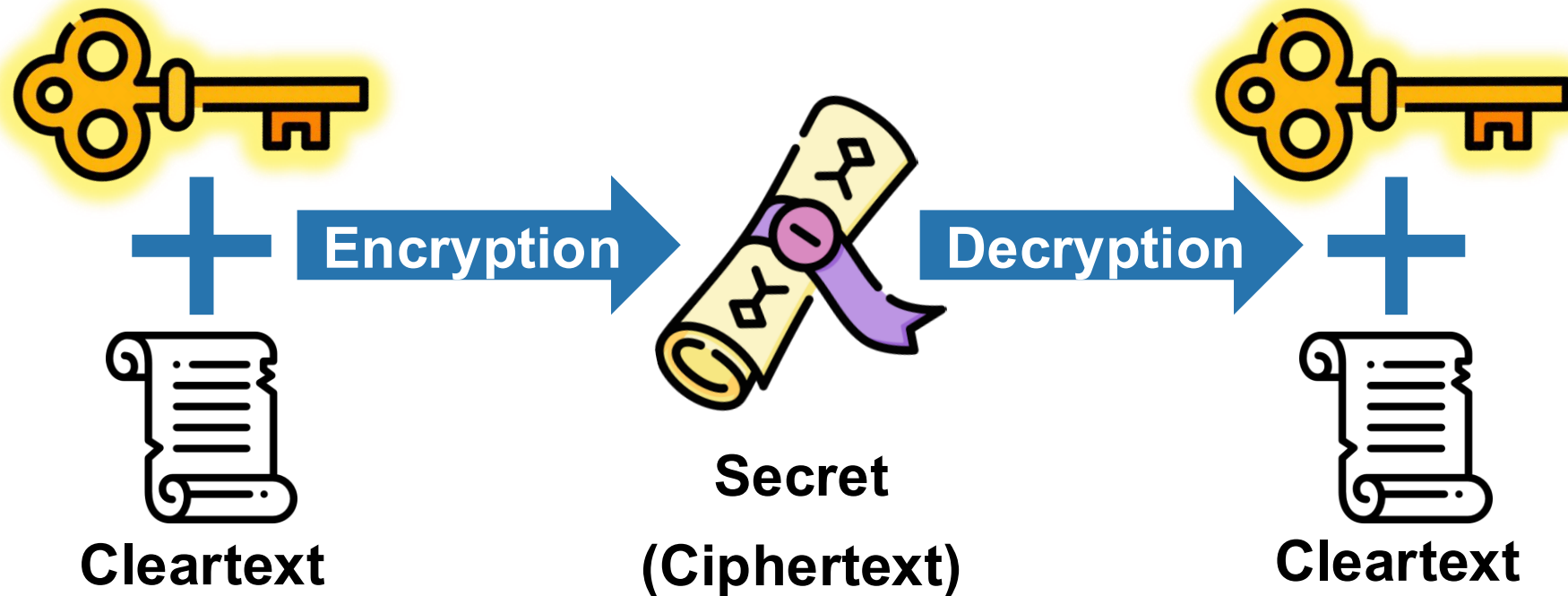
- When proposing a **New Security Approach** or Algorithm

Complementary Obscurity

Hide implementation details & configuration details (e.g., version of a particular framework, data base, server configuration) to make it **harder** to exploit known vulnerabilities

- When implementing a commercial system by using existing, publicly scrutinized technology

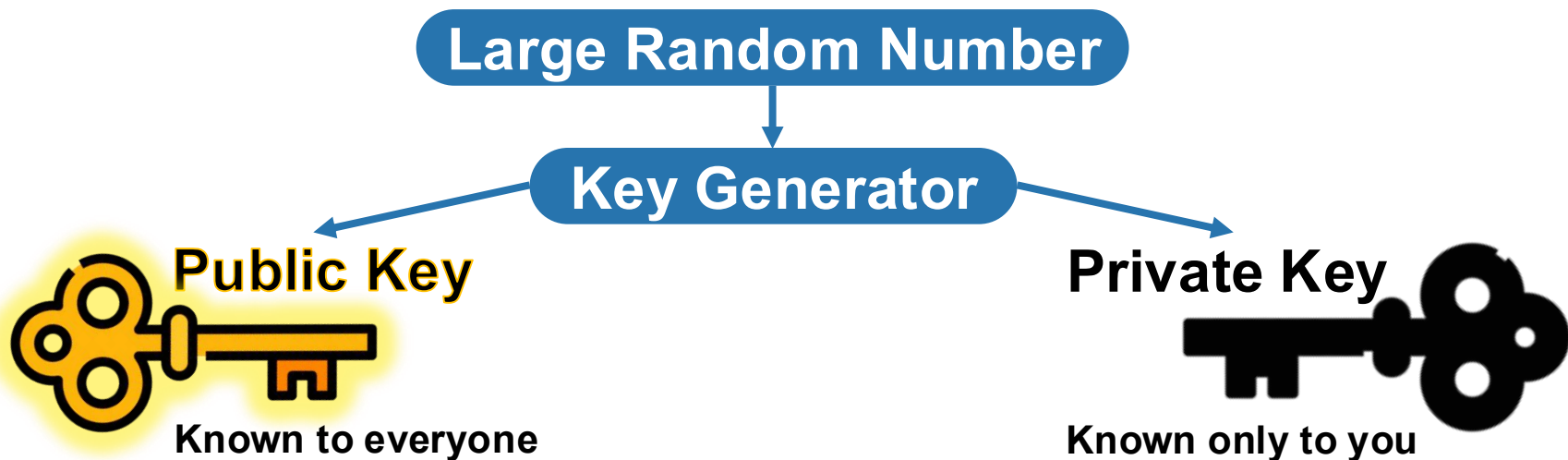
Symmetric Encryption (e.g., AES) Ensures Confidentiality



Public-Key Cryptography (e.g., RSA)

Goal

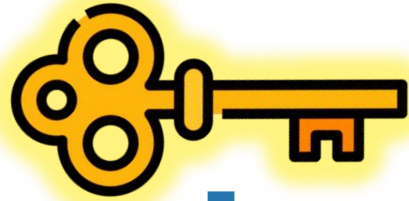
How to encrypt & decrypt data **without having to secretly exchange a key?**



To Send a Message to Bob, Alice Encrypts the Message with Bob's Public Key

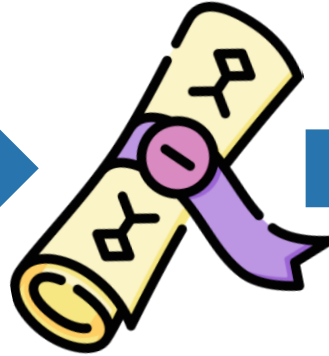
Bob's Public Key

Bob's Private Key



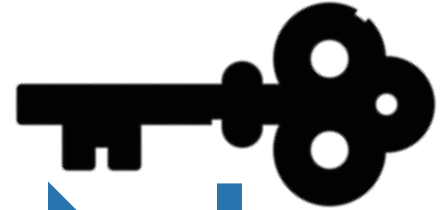
+

Encryption



Bob's Secret
(Ciphertext)

Decryption



+



Cleartext

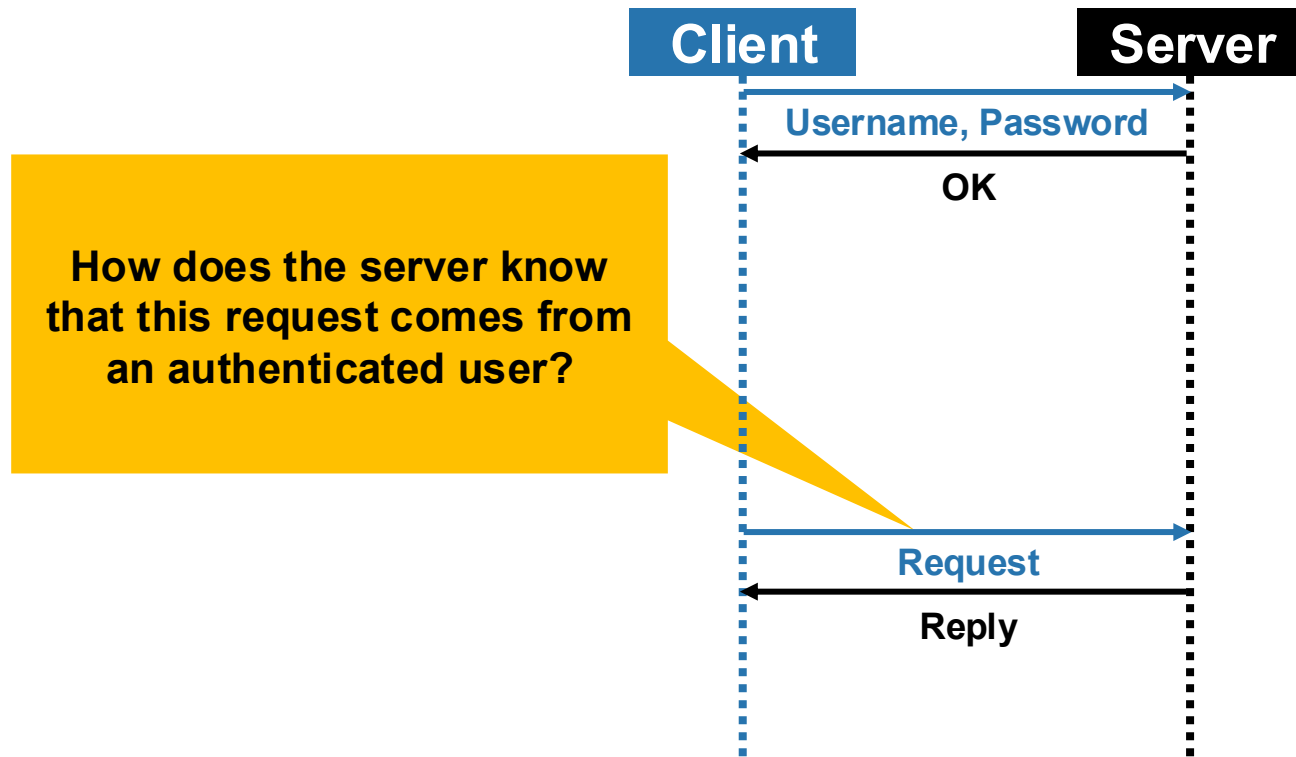
Cleartext

Digital Signatures

- **Encrypting** a document with your **private key** allows everyone to read it (by decrypting it with your **public key**)
- Because only you have your **private key**, you are the only one who **could have written it**
- Encrypting & decrypting the entire document is slow
 - Encrypting just the hash of the document is faster



Authentication is Not Trivial



Naïve Approach: Always send username & password?

Don't do this!

Goal

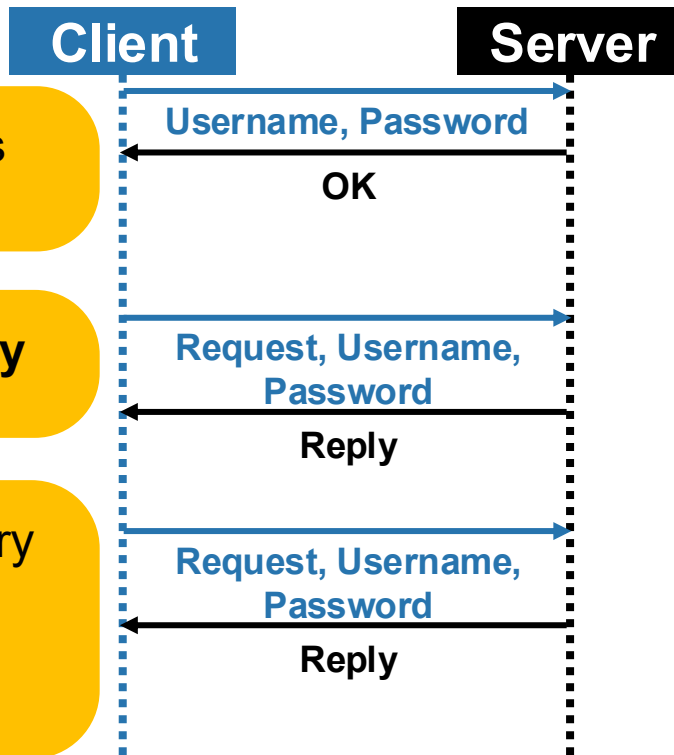
How to prove to the server that the request comes from an authenticated user?

Solution

Client sends **username** and **password with every request**. Server verifies them with every request.

Consequences

- **Slow** (server needs to verify the password every time)
- **Insecure** (Client needs to memorize the password in memory)



Session-Based Authentication via Session Cookies

Goal

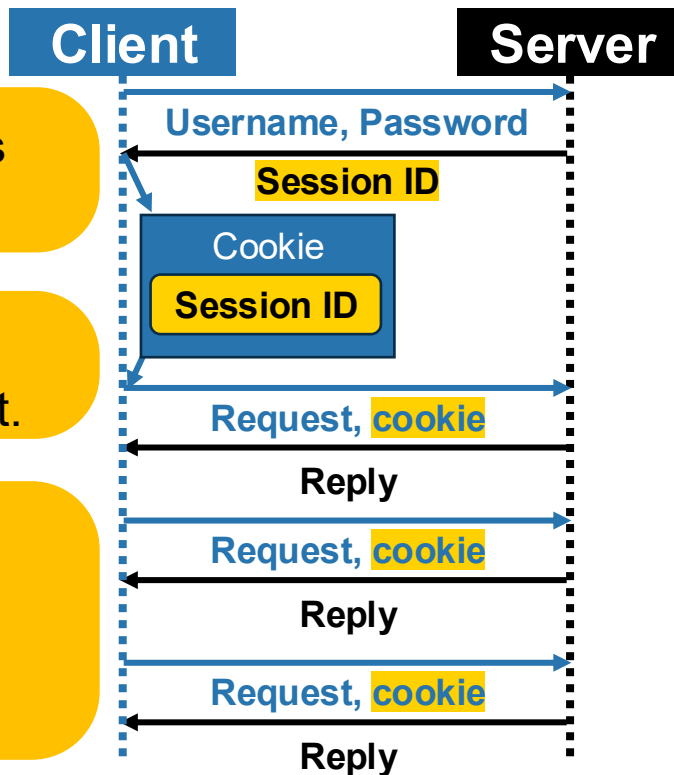
How to prove to the server that the request comes from an authenticated user?

Solution

After initial authentication server generates a session ID that the client sends with every request.

Details

- A new session ID for each session
- Session ID is stored in a client-side cookie (set `HttpOnly` flag, which prevents client-side JS from accessing the cookie)



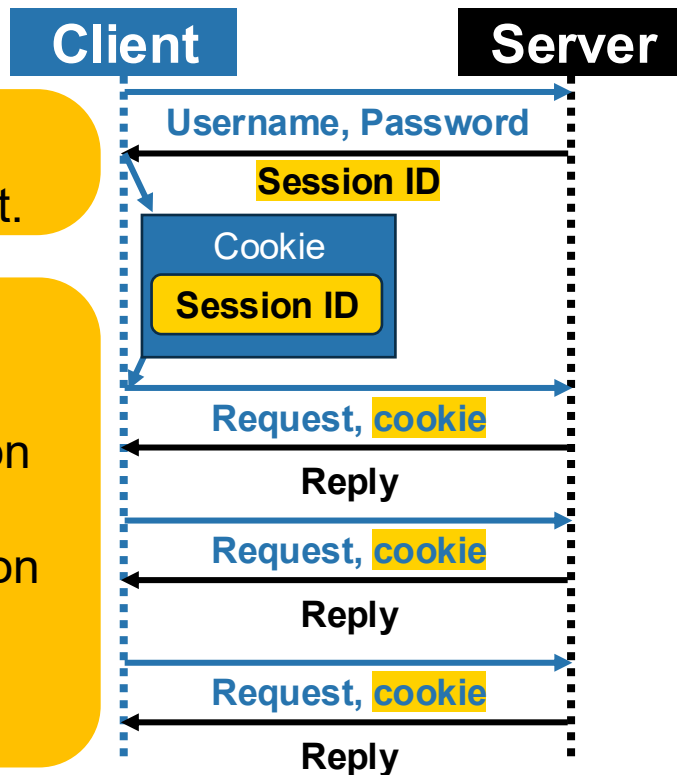
Session-Based Authentication via Session Cookies

Solution

After initial authentication server generates a session ID that the client sends with every request.

Consequences

- **Fast** (no database operations needed to verify authentication)
- **Stateful** (the server needs to maintain a session ID with the corresponding ID)
- **Somewhat Secure** (session has short expiration time)
- **Still vulnerable to XSS attacks** (JS code can ride on the session)



Authentication via JSON Web Token (JWT)

Goal

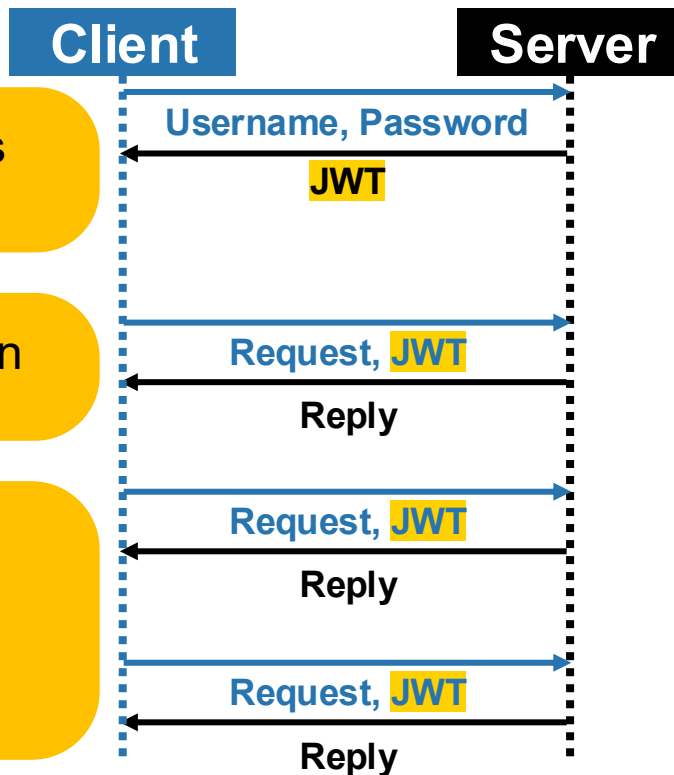
How to prove to the server that the request comes from an authenticated user?

Solution

After initial authentication server generates a token (JWT) that the client sends with every request.

Details

- JWT is an identifier of the user (e.g., user ID) **signed by the server's private key**
- Stored **client-side** in memory, in cookies (HTTPOnly & SameSite), or local storage



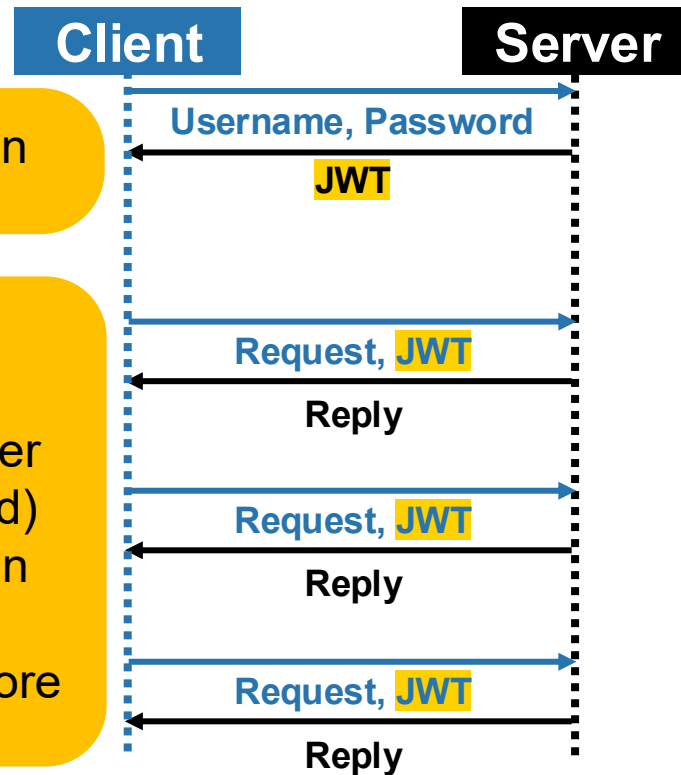
Authentication via JSON Web Token (JWT)

Solution

After initial authentication server generates a token (JWT) that the client sends with every request.

Consequences

- **Fast** (no database operations needed to verify authentication)
- **Somewhat Secure** (JWT is signed by the server & has shorter expiration time than the password)
- **Vulnerable to XSS attacks** (Since the JWT can be accessed via JS code)
- **Statelessness** (the server does not need to store a session ID)



A Security Plan Analyzes Four Aspects of the System

Security Model

What are you defending?

Threat Model

Who might be attacking? What is the attacker trying to achieve?

Attack Surface

Which parts of the system are exposed to an attacker?

Protection Mechanisms

How do we prevent an attacker from compromising the system?

Build a Threat Model

Threat Model

Who might be attacking? What is the attacker trying to achieve?

- **Knowledge:** What does the attacker know?
- **Actions:** What can the attacker do?
- **Resources:** How much effort can it spend?
- **Incentive:** Why does the attacker want to do this?

What is the issue here?

Wrong threat model

Phone numbers with 9s and 1s
can be dialed.

Larger Attack Surface

Foil can be pushed down to dial
other buttons



Security Design Principle: Principle of Least Privilege

Goal

How to **reduce the attack surface** and the impact of a security breach (i.e., **blast radius**)?

Principle of Least Privilege

(aka Principle of Minimal Privilege / Principle of Least Authority)

Every **program** and every privileged **user** of the system should **operate using the least set of privileges necessary to complete the job**.

Implementation Ideas

all user accounts / processes should run with as few privileges as possible & launch applications with as few privileges as possible

Split your Application into Separate Components & Apply the Principle of Least Privilege

If an attacker compromises one component, the impact is reduced

Read-only access to the Products database table.

Product Display Service

Email Notification Service

Access to a **specific email sending API** (e.g., SendGrid, AWS SES) and **no database** access.

Write access to a specific folder on the cloud storage bucket (e.g., /uploads) and **no delete** permission.

Image Upload Service

System Backup Service

Read-only access to the file system or database. **Write access** only to the designated backup storage location.

Please fill out your Exit Tickets on Bruin Learn!

Question 1

1 pts

In your own words, please summarize **three key insights** you learned about **Security & Authentication** today. (Do not copy phrases from the lecture slides)

Question 2

1 pts

For **each** of the **three** security attributes of the CIA triad, describe one **potential violation** of the attribute that would be problematic for your course **project** system.

Question 3

Please leave any questions that you have about today's material and things that are still unclear or confusing to you (if none, simply write N/A)

Credits: These slides use images from Flaticon.com (Creators: Freepik, monkik, kliwir-art, md-tanvirul-haque, metami-septiana)